

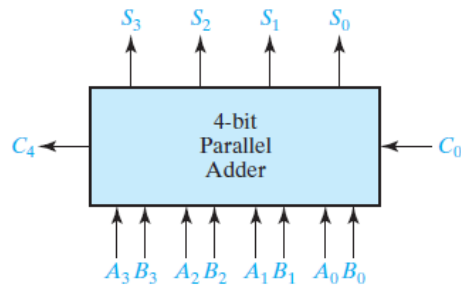
BECE102L – Digital Systems Design

Module 4 – Design of Data Path Circuits

Dr. Prayline Rajabai C
School of Electronics Engineering,
Vellore Institute of Technology, Vellore

Design of 4-bit parallel binary adder

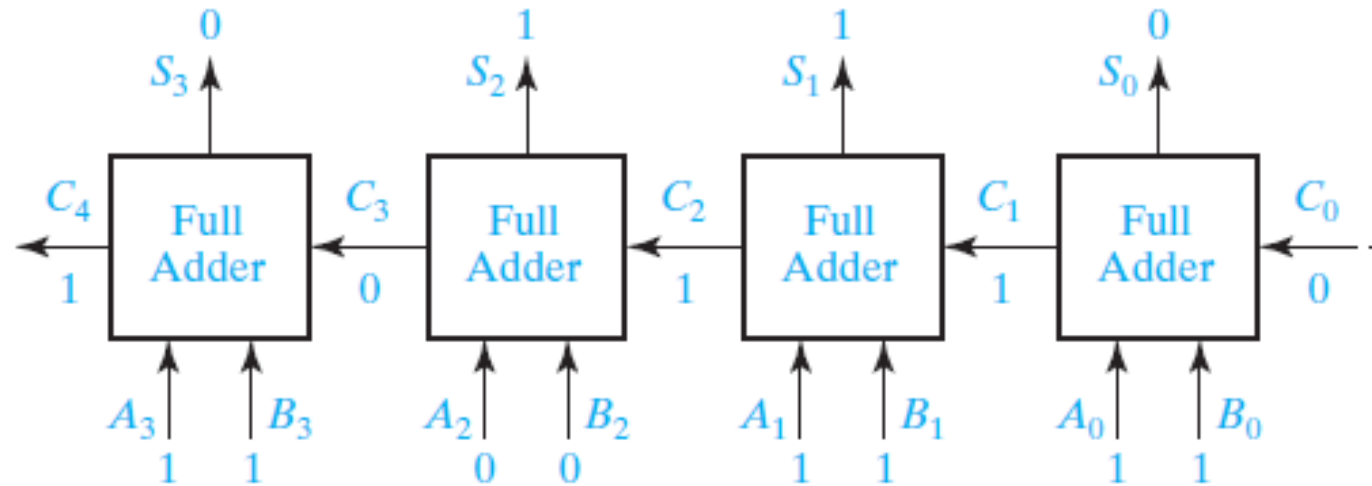
- A parallel adder that adds two N-bit unsigned binary numbers and a carry input to give a N-bit sum and a carry output.
- If $N = 4$, then it is 4-bit parallel binary adder.



- One approach would be to construct a truth table with nine inputs and five outputs and then derive and simplify the five output equations.
- But, each equation would be a function of nine variables before simplification and this approach would be very difficult, and the resulting logic circuit would be very complex.

Design of 4-bit parallel binary adder

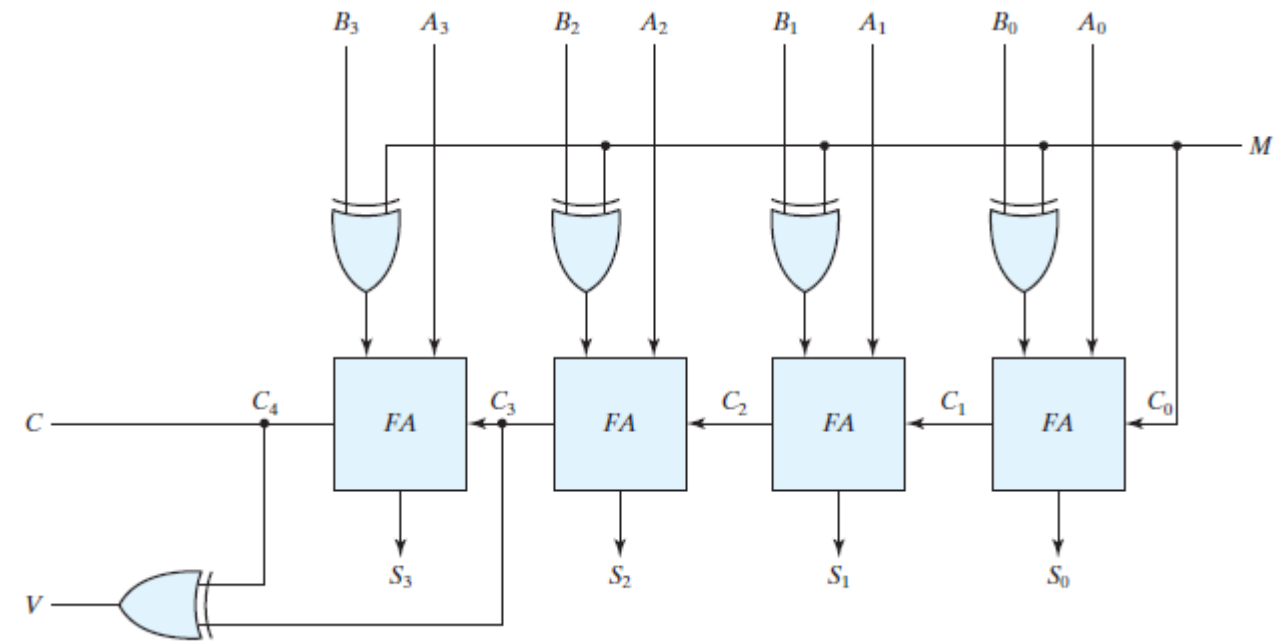
- A better method is to design a logic module that adds two bits and a carry, and then connect four of these modules together to form a 4-bit adder.



- If the number of bits is large (N is large), a parallel binary adder of the type may be rather slow because the carry generated in the first cell might have to propagate all of the way to the last cell.
- The number of gate levels for the carry propagation can be found from the circuit of the full adder.

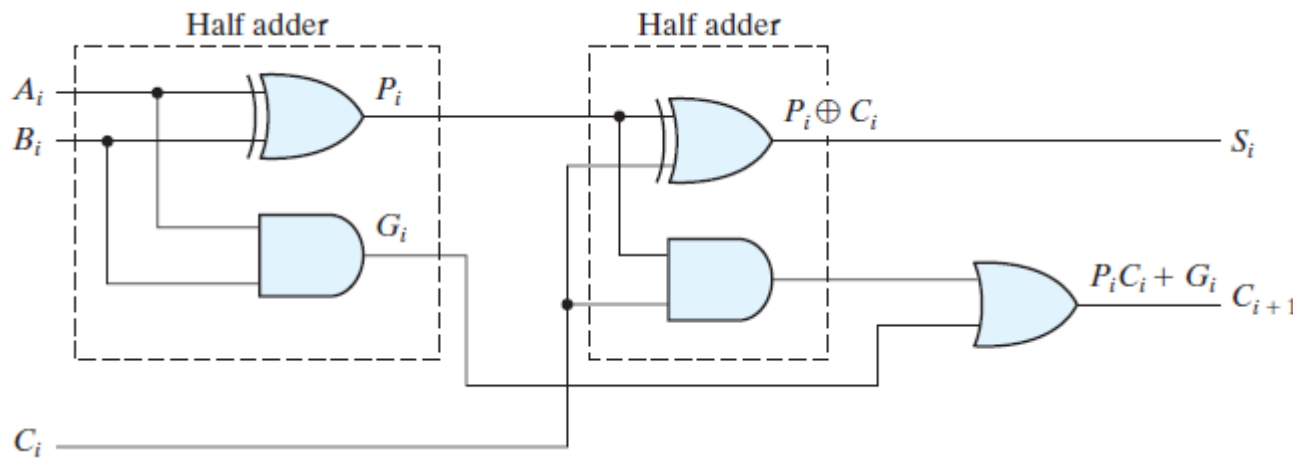
Four-bit adder-subtractor

- The subtraction of unsigned binary numbers can be done most conveniently by means of complements.
- Remember that the subtraction $A - B$ can be done by taking the 2's complement of B and adding it to A .
- The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant pair of bits.
- The 1's complement can be implemented with inverters
- 1 can be added to the sum through the input carry.



Full adder with P and G

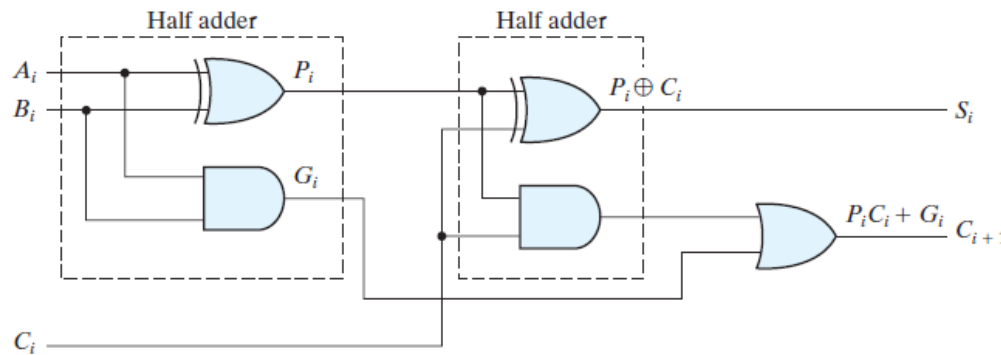
- A parallel adder that adds two N-bit unsigned binary numbers and a carry input to give a N-bit sum and a carry output.



- The signal from the input carry C_i to the output carry C_{i+1} propagates through an AND gate and an OR gate, which constitute two gate levels.
- If there are four full adders in the adder, the output carry C_4 would have $2 * 4 = 8$ gate levels from C_0 to C_4 .
- For an n - bit adder, there are $2 n$ gate levels for the carry to propagate from input to output.

Carry Lookahead

- There are several techniques for reducing the carry propagation time in a parallel adder. The most widely used technique employs the principle of carry lookahead logic



$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

- G_i is called a carry generate, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i .
- P_i is called a carry propagate, because it determines whether a carry into stage i will propagate into stage $i + 1$.

Carry Lookahead

Carry Lookahead Logic

$$C_0 = \text{input carry}$$

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 = P_2P_1P_0C_0$$

$$P_i = A_i \oplus B_i$$

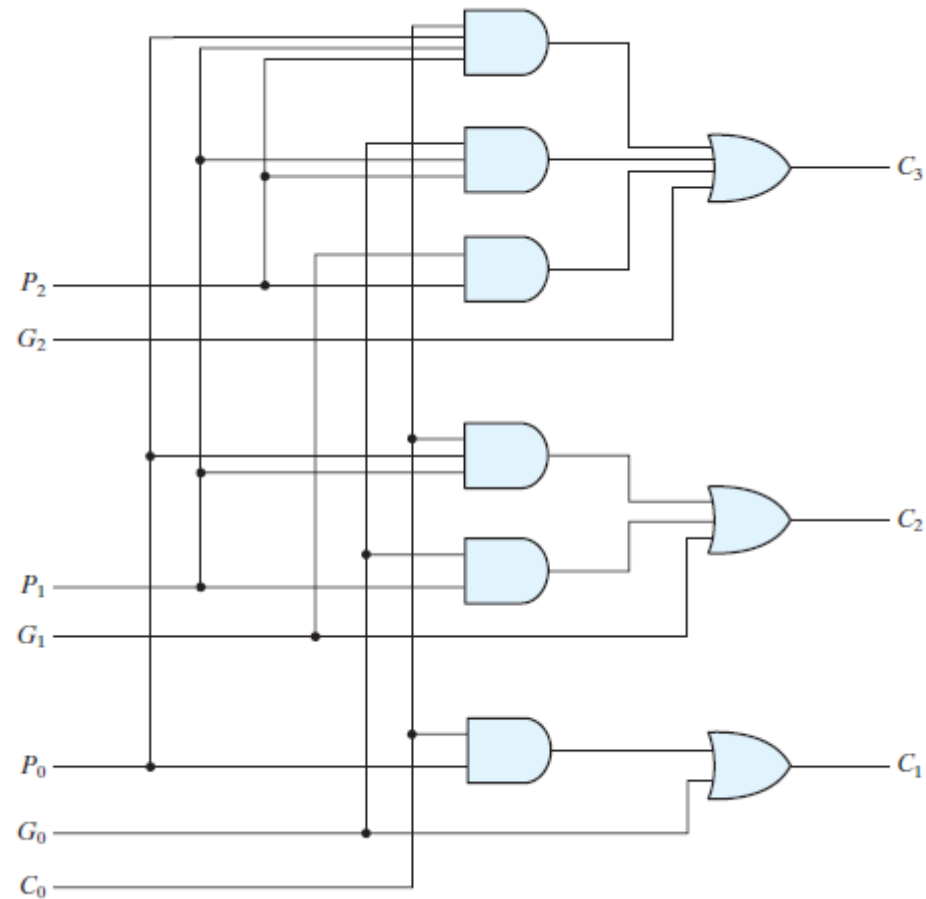
$$G_i = A_iB_i$$

$$S_i = P_i \oplus C_i$$

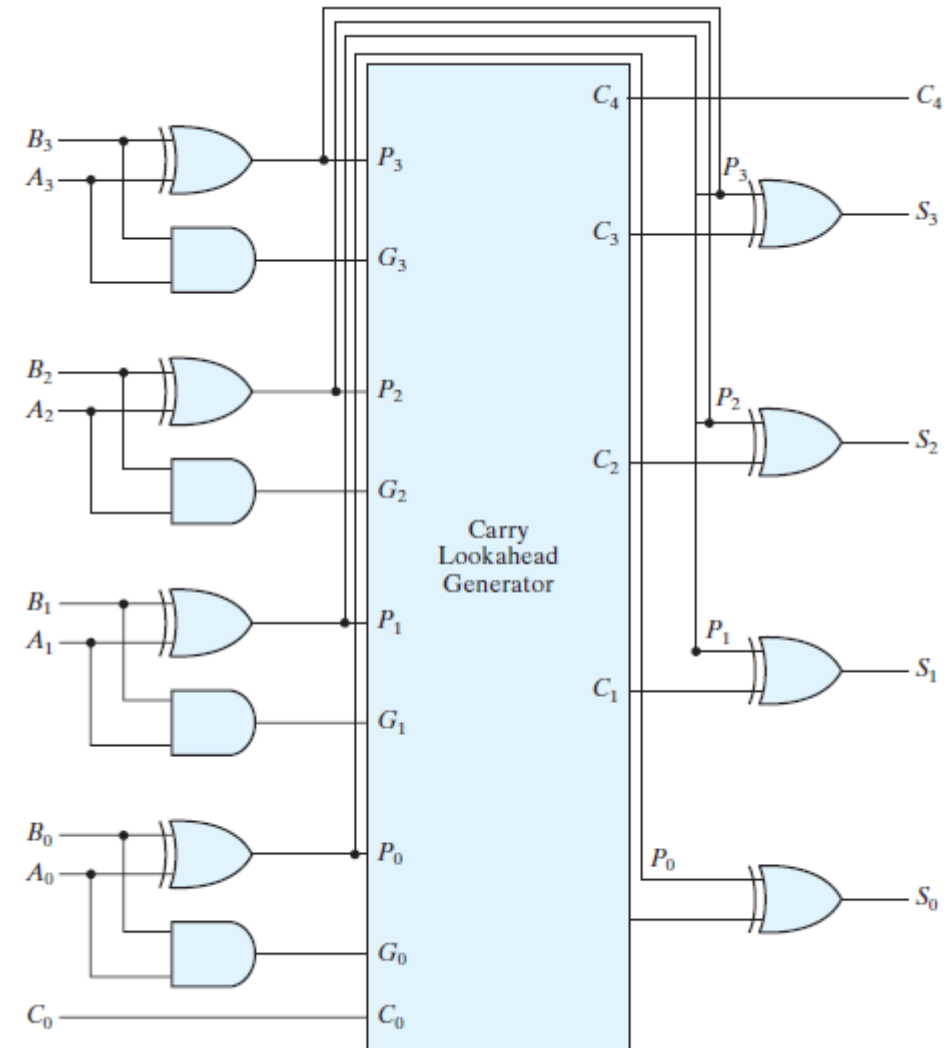
$$C_{i+1} = G_i + P_iC_i$$

- The three Boolean functions for C_1 , C_2 , and C_3 are implemented in the carry lookahead generator
- All output carries are generated after a delay through two levels of gates.
- Thus, outputs S_1 through S_3 have equal propagation delay times.

Carry Lookahead Adder



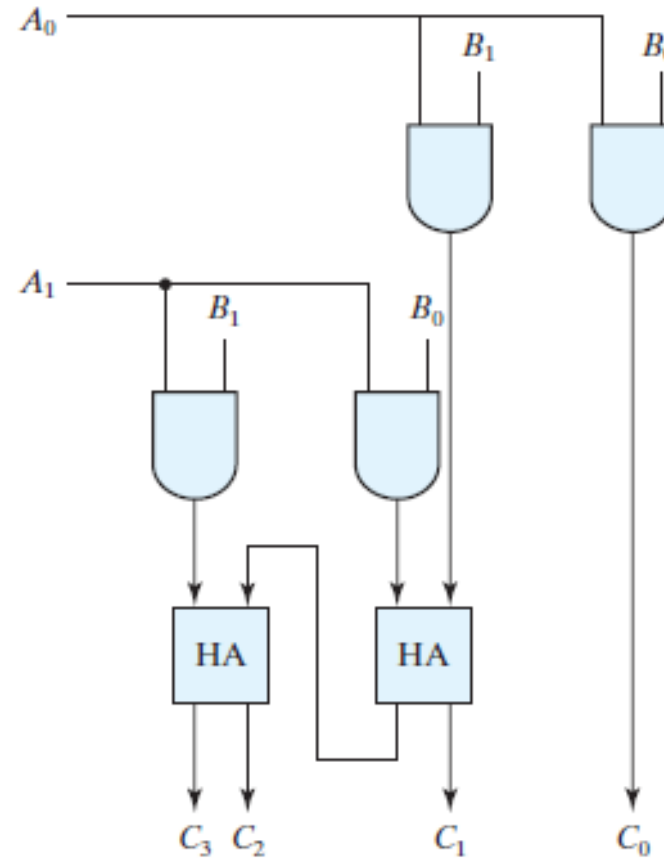
Logic diagram of carry lookahead generator



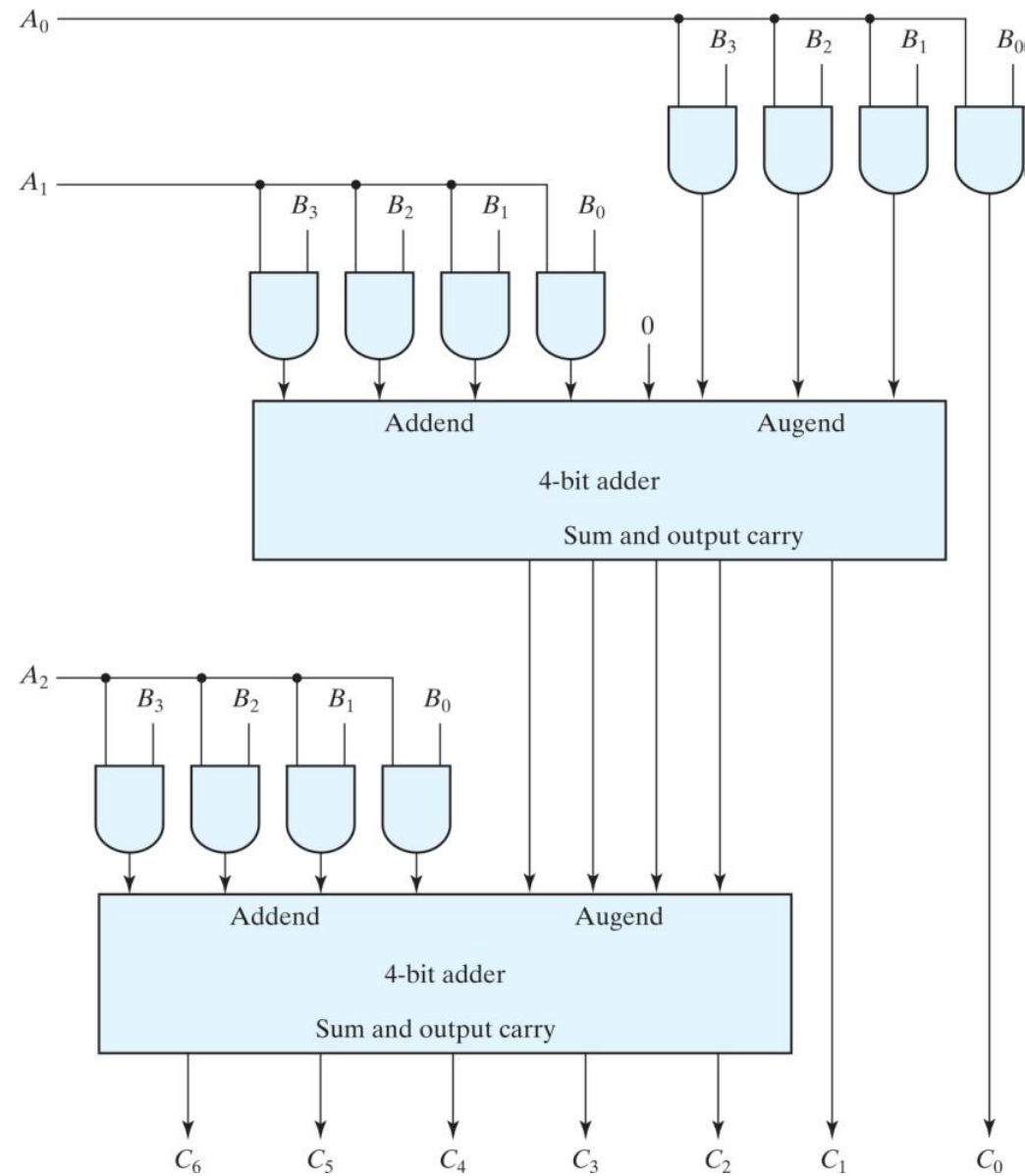
Four-bit adder with carry lookahead

2x2 Binary Multiplier

		B_1	B_0
		A_1	A_0
		A_0B_1	A_0B_0
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0



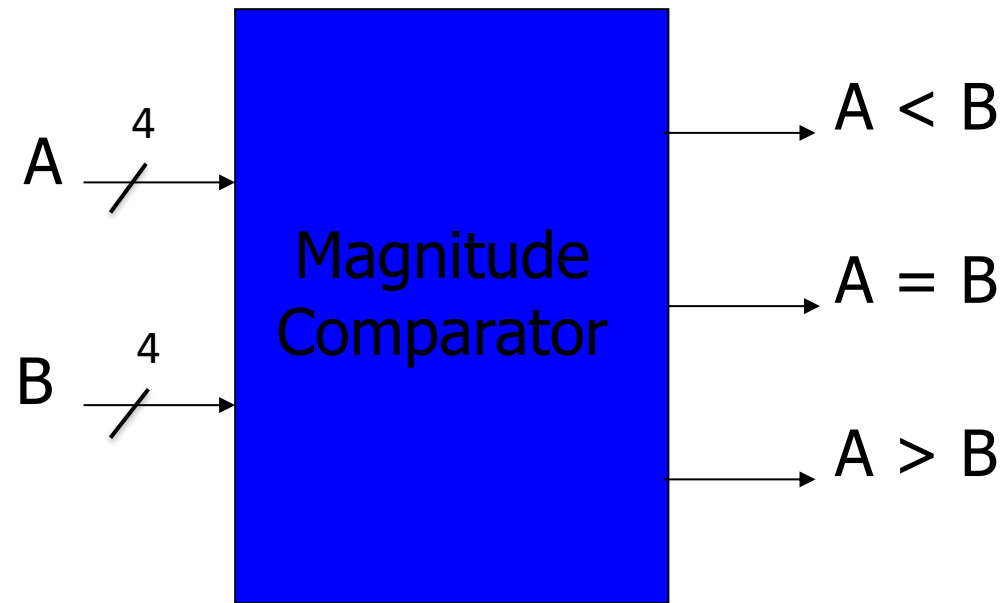
4x3 Binary Multiplier



Magnitude Comparator

- Magnitude comparators compare two multi-bit binary numbers
- Magnitude comparators allow for data comparison and can be built using and-or gates
- Greater/less than function requires more hardware than equality
- The comparison of two numbers gives the outputs: $A > B$, $A = B$, $A < B$
- Design Approaches
 - Truth table : 2^{2n} entries - too cumbersome for large n
 - use inherent regularity of the problem
 - reduce design efforts
 - reduce human errors

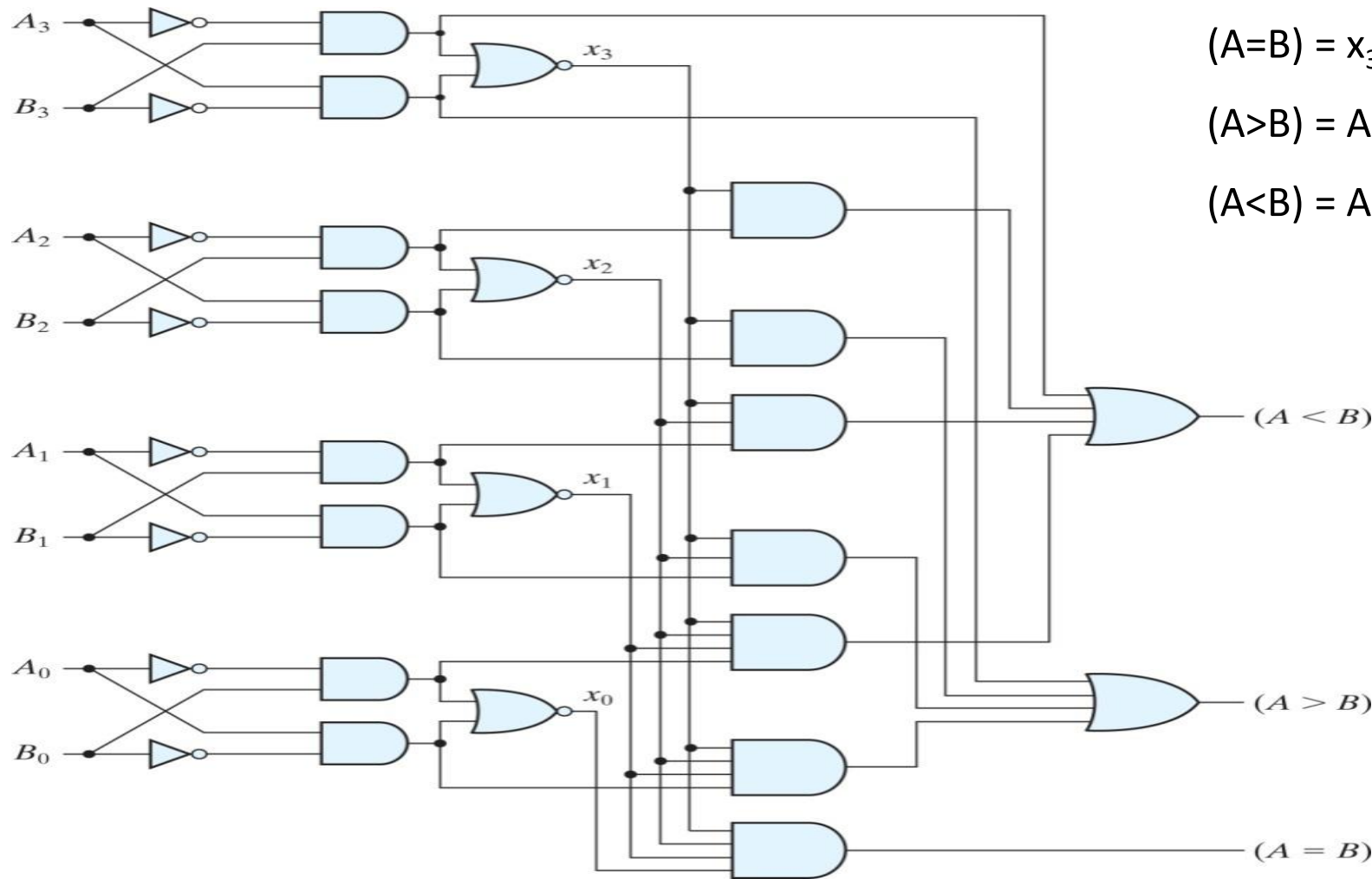
Block Diagram of Magnitude Comparator



Magnitude Comparison

- Algorithm -> logic
 - $A = A_3A_2A_1A_0$; $B = B_3B_2B_1B_0$
 - $A=B$ if $A_3=B_3, A_2=B_2, A_1=B_1$ and $A_0=B_0$
- Test each bit:
 - equality: $x_i = A_iB_i + A_i'B_i' = (A_iB_i' + A_i'B_i)'$
 - $(A=B) = x_3x_2x_1x_0$
- More difficult to test less than/greater than
 - $(A>B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$
 - $(A<B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$
 - Start comparisons from **high-order** bits
- Implementation
 - $x_i = (A_iB_i' + A_i'B_i)'$

Logic Circuit for Magnitude Comparison



$$(A=B) = x_3x_2x_1x_0$$

$$(A>B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$$

$$(A<B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$$

